Part 1

Row Key & Sharding: In Bigtable, data is dynamically partitioned (sharded) across nodes based on the Row Key. What is your proposed Row Key for this dataset? Explain how this choice avoids "hotspotting" and impacts your ability to efficiently query vehicles by *City* or *Make*.

For the primary Row Key, I would use `reverse_vin#vin`. Reversing the VIN can help spread writes across different tablets instead of having them concentrated in one area. The VIN is still included so each vehicle can be identified. However, querying by fields like City or Make would be less efficient because these fields aren't part of the row key. Bigtable would need to scan the table to find matching vehicles unless separate lookup tables were created, such as `city#vin` or `make#vin`, that point back to the main records.

Column Families: How would you design the Column Families for this dataset? Group the attributes logically and explain how this aids in distributed read performance.

The dataset attributes should be split into three logical column families based on access patterns: `veh` for core vehicle specs (`make, model, year, type, range, cafv`), `loc` for spatial attributes (`county, city, state, zip, dist, coords, geoid`), and `meta` for administrative fields (`dol_id, utility`). This keeps related attributes together and can make reads more efficient. For example, a query calculating the average electric range by manufacturer would only need the `veh` column family, so it wouldn't have to read location data or other fields that aren't needed for the query.

Horizontal Scaling: Compare the pros and cons of using a *Hashed Shard Key* (e.g., on VIN) versus a *Range-Based Shard Key* (e.g., on Model Year) regarding query routing efficiency and cluster performance.

The main trade-off between hashed and range-based shard keys is how they distribute writes and handle range queries. A hashed key, such as a hash of the VIN, spreads records across the cluster and helps prevent write hotspots. However, queries for a range of model years, such as 2010 to 2020, become less efficient because the matching records may be spread across multiple tablets. A range-based key using Model Year makes these queries easier because records from similar years are stored together. However, if many new registrations target the same range of years, the tablets holding those records could receive more traffic than others. For this dataset, I would use a hashed key for the main table and create separate lookup tables for queries by fields like Model Year, City, or Make. This would help distribute writes while still making those queries more efficient.

Part 4

Using a tool like Postman, curl, or a simple Python script, measure the response time of making 500 requests to /insert-fast vs. 500 requests to /insert-safe. Report the average latency for both. Based on your latency numbers and the CAP theorem, give a real-world scenario where you would intentionally choose the faster, primary-only write preference despite the risk of data loss.

To test the performance difference, I sent 500 POST requests to both endpoints from the VM. The fast endpoint (`w=1`) averaged 18.26 ms with a median of 4.00 ms, while the safe endpoint (`w="majority"`) came out to an average of 20.05 ms with a median of 6.54 ms.

Looking at the medians instead of the means can be a more fair comparison. That shows the majority write was only about 2.5 milliseconds slower per request. This lines up with how MongoDB works: `w=1` finishes as soon as the primary node takes the write, whereas `w="majority"` forces it to wait for at least one secondary node to confirm replication before responding.

For when you'd actually want to use the faster, riskier write preference, a great real-world example is handling high-volume telemetry or IoT data, like GPS pings from vehicles. In systems like that, data is streaming in constantly every couple of seconds. If the primary node crashes right after a write and a single ping gets lost, it's not a critical loss because the next update will already come soon.